# Immutable Data Types

Final fields, records, enums, and the Comparable interface

The first set of notes built a `Student` class whose percent could change through a setter. This set asks a different design question: what if an object could never change at all? We will build that idea up in steps, from `final` fields, to a hand-written immutable class, to records, which do most of the work for you. Then we look at enums, which are classes with a fixed set of immutable objects, and finish with the `Comparable` interface, which lets Java sort both. For reading, see Core Java, Volume I, Chapter 4 (final fields and records), Chapter 5 (enumeration classes), and Chapter 6 (interfaces, including `Comparable`).

**A note before you begin.** These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

## 1. Mutable and Immutable Objects

An object is *mutable* if its state can change after it is created. The `Student` from part 1 is mutable, because `setPercent` changes it. An object is *immutable* if its state is fixed the moment the constructor finishes.

You have used an immutable class since your first program: `String`. No `String` method changes the string it is called on. Methods that look like they change it actually return a new string.

```
String x = "ada";
String y = x.toUpperCase();
System.out.println(x + " " + y);   // ada ADA   (x did not change)
```

Why design a class this way?

- **It is safe to share.** You can hand an immutable object to any method or store it in any list, and nobody can change it behind your back. You never need a defensive copy.
- **It is easy to reason about.** If the constructor checks that the data is valid, the object stays valid forever. There is no setter that can break it later.
- **It works safely as a key.** If you put a mutable object into a `HashSet` or use it as a `HashMap` key and then change its fields, its hash code changes and the collection can no longer find it.

Immutable objects cannot cause that problem.

- **It is safe across threads.** When you get to concurrent programming in later courses, immutable objects need no locking.

The cost is that every "change" creates a new object. For small data objects that cost is almost always worth paying.

## 2. The final Keyword

A `final` variable can be assigned only once. A `final` field must be assigned exactly once, by the time the constructor finishes (or, for a `static final` field, by the time the static initialization is done, as you saw with `Config` in part 1). The compiler enforces this on every path through the constructor.

The single most important thing to understand about `final` is what it does *not* do. It locks the *variable*, not the *object* the variable refers to.

```
final ArrayList<String> names = new ArrayList<>();
names.add("Ada");             // allowed: the list changes, the variable does not
names = new ArrayList<>();    // not allowed
```

```
error: cannot assign a value to final variable names
```

So a `final` field that holds a list, an array, or any other mutable object does not make the class immutable by itself. We deal with that in the recipe below.

The word `final` has two more uses that belong to inheritance: a `final` method cannot be overridden, and a `final` class cannot be extended. We will return to both in the next unit.

## 3. A Recipe for an Immutable Class

1. Make every field `private final`.
2. Provide no setters or any other method that changes a field.
3. Validate the data in the constructor, so a bad object can never exist.
4. Make the class `final`, so nobody can write a subclass that adds mutable behavior. (More on this in the inheritance unit.)
5. If a field holds a mutable object, such as a list, store a copy that cannot be changed, and never hand out the original.
6. For a "change," write a method that returns a new object, often named `withSomething`.

Here is `Student` rewritten as an immutable class.

```
public final class Student {
    private final String id;
    private final String name;
    private final double percent;
```

```
    public Student(String id, String name, double percent) {
        if (percent < 0 || percent > 100) {
            throw new IllegalArgumentException("percent must be 0 to 100: " + percent);
        }
        this.id = id;
        this.name = name;
        this.percent = percent;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getPercent() { return percent; }

    // No setter. A "change" produces a new object instead.
    public Student withPercent(double newPercent) {
        return new Student(id, name, newPercent);
    }
}
```

```
Student s = new Student("S1", "Ada", 88.0);
Student t = s.withPercent(91.5);
System.out.println(s.getPercent() + " " + t.getPercent());   // 88.0 91.5
```

This class works, but notice how much of it is boilerplate. If we also wrote `equals`, `hashCode`, and `toString`, which a data class like this should have, it would be several times longer. Records exist to remove that boilerplate.

## 4. Records

A *record* is a class whose whole job is to carry a fixed set of values. Here is the immutable `Student` as a record:

```
public record Student(String id, String name, double percent) { }
```

The list in parentheses declares the record's *components*. From that one line, Java generates everything in the table below.

**Table 1:** What Java generates for the record declaration above.

| Generated member | What it does |
|---|---|
| **Fields** | One `private final` field per component: `id`, `name`, `percent` |
| **Canonical constructor** | `Student(String id, String name, double percent)`, which assigns all three fields |
| **Accessor methods** | `id()`, `name()`, and `percent()`; note there is no `get` prefix |
| `equals` | Two records are equal when all of their components are equal |
| `hashCode` | Computed from all components, consistent with `equals` |
| `toString` | Prints the record name and every component |

```
Student ada = new Student("S1", "Ada", 91.5);
System.out.println(ada);                        // Student[id=S1, name=Ada, percent=91.5]
System.out.println(ada.name());                 // Ada
Student copy = new Student("S1", "Ada", 91.5);
System.out.println(ada.equals(copy));           // true: same values
System.out.println(ada == copy);                // false: two different objects
```

## The compact constructor

To validate or clean up the data, write a *compact constructor*. It has no parameter list, because the parameters are the components. You check and adjust the parameters, and Java assigns the fields for you when the body finishes. Do not write `this.name = name` yourself.

```
public record Student(String id, String name, double percent) {
    public Student {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        if (percent < 0 || percent > 100) {
            throw new IllegalArgumentException("percent must be 0 to 100: " + percent);
        }
        name = name.strip();   // adjusts the parameter; the field is set from it
    afterward
    }
}
```

Now `new Student("S1", " Ada ", 91.5)` stores the name as `Ada`, and `new Student("S9", "Bob", 140)` throws an exception with the message `percent must be 0 to 100:  140.0`. Every `Student` that exists is valid.

## Adding methods and static members

A record body can contain instance methods, static fields, and static methods. This is where part 1 comes back: a static counter and a static *factory method* can hand out IDs.

```
    private static int nextId = 1;

    // Static factory that hands out the next ID.
    public static Student create(String name, double percent) {
        String id = "S" + nextId;
        nextId++;
        return new Student(id, name, percent);
    }

    public LetterGrade letterGrade() {
        return LetterGrade.fromPercent(percent);
    }

    public Student withPercent(double newPercent) {
        return new Student(id, name, newPercent);
```

```
    }
```

`LetterGrade` is the enum in the next section. The complete record, with everything assembled, appears in the `Comparable` section.

### What a record cannot do

- **No extra instance fields.** All instance state must be in the component list. Declaring another instance field gives `error: field declaration must be static`. Static fields are fine.
- **No setters in disguise.** The fields are `final`, so no method can change them.
- **No extending another class.** A record cannot extend a class, and no class can extend a record. A record *can* implement interfaces, which is how it will become `Comparable` below.

### Records are only as immutable as their components

A record stores references. If a component is a mutable object, such as an `ArrayList`, outside code can still change that list. The fix is step 5 of the recipe: store an unmodifiable copy in the compact constructor.

```
import java.util.List;

public record Section(String code, List<Student> students) {
    public Section {
        students = List.copyOf(students);   // unmodifiable snapshot
    }
}
```

After this, clearing the original list does not affect the section. Trying to add to the section's list with `section.students().add(...)` fails with an exception: `UnsupportedOperationException`.

### Records are only as equal as their components

The generated `equals` does not compare components with `==`. For each component that is an object, it calls that component's own `equals` method, and it compares primitive components such as `double` by value. That works well for `String`, the wrapper classes, `List`, enums, and other records, because they all override `equals` to compare contents. It goes wrong when a component's type does not override `equals`. Then the call falls back to the version inherited from `Object`, which only checks whether two references point to the same object.

Arrays are the most common surprise, because arrays never override `equals`:

```
record Scores(String name, int[] values) { }

Scores a = new Scores("Ada", new int[] {90, 85});
Scores b = new Scores("Ada", new int[] {90, 85});
System.out.println(a.equals(b));   // false: two different array objects
// The next line prints something like: Scores[name=Ada, values=[I@65ab7765]
System.out.println(a);
```

The same thing happens with a component from a class you wrote without an `equals` method. `hashCode` and `toString` follow the same rule, which is why `toString` printed a type code and hash code instead of the numbers.

**Components of type double.** A record compares `double` components the way `Double.compare` does, not the way `==` does. For ordinary numbers the two agree, but they disagree on two special values: `NaN` ("not a number," the result of `0.0 / 0.0`) and negative zero.

```
record Reading(double value) { }

double nan = Double.NaN;
System.out.println(nan == nan);                                    // false
System.out.println(new Reading(nan).equals(new Reading(nan)));     // true
System.out.println(0.0 == -0.0);                                   // true
System.out.println(new Reading(0.0).equals(new Reading(-0.0)));    // false
```

You will rarely meet either value, but rounding is a more common surprise. A record holding `0.1 + 0.2` is not equal to one holding `0.3`, because the two `double` values differ in their last digits. Comparing computed `double` values for exact equality is risky whether or not they are inside a record.

To stay safe:

- Choose component types that compare by content: `String`, the wrapper classes, enums, records, and `List`.
- Use a `List` instead of an array. If you must use an array, write `equals`, `hashCode`, and `toString` yourself using `Arrays.equals`, `Arrays.hashCode`, and `Arrays.toString`.
- Keep components unchanging, as the `Section` record does with `List.copyOf`. A record holding a list that can still change will change its hash code when the list changes, so a `HashSet` that already contains the record can no longer find it.
- Be cautious about relying on `equals` for records whose `double` components come from calculations.

## Records and the file formats you already know

Each row of a CSV file, or each object in a JSON array, is a fixed group of values, which is exactly what a record represents. Reading a line such as `S1,Ada,91.5` becomes:

```
String[] parts = line.split(",");
Student s = new Student(parts[0], parts[1], Double.parseDouble(parts[2]));
```

If the file contains a bad percent, the compact constructor rejects it at the moment of reading, instead of letting a bad object travel through the rest of the program.

**Record or class?** Use a record when the object is plain data whose identity is its values. Use an ordinary class when the object needs to change over time, needs hidden state that is not part of its value, or will be part of an inheritance hierarchy.

## 5. Enums as Full Classes

You have probably written simple enums such as `enum Day { MONDAY, TUESDAY }`. An enum is really a class with a fixed set of instances, created once, when the enum is first used. Because it is a class, it can have fields, a constructor, and methods.

Here is an enum for this course's letter grades, using the cutoffs from the syllabus grade scale.

```
public enum LetterGrade {
    A(89.50, 4.0),
    B(79.50, 3.0),
    C(69.50, 2.0),
    D(59.50, 1.0),
    F(0.00, 0.0);

    private final double minPercent;
    private final double gradePoints;

    LetterGrade(double minPercent, double gradePoints) {
        this.minPercent = minPercent;
        this.gradePoints = gradePoints;
    }

    public double getMinPercent() { return minPercent; }
    public double getGradePoints() { return gradePoints; }

    // Relies on the constants being declared from highest cutoff to lowest.
    public static LetterGrade fromPercent(double percent) {
        for (LetterGrade g : values()) {
            if (percent >= g.minPercent) {
                return g;
            }
        }
        throw new IllegalArgumentException("percent below zero: " + percent);
    }
}
```

Points to notice:

- `A(89.50, 4.0)` calls the constructor. Each constant is one object, built once.
- The constructor is automatically `private`. No code can write `new LetterGrade(...)`, so the five constants are the only `LetterGrade` objects that will ever exist.
- The fields are `final`, so the enum is immutable. Keep enum fields `final`; a mutable field on an enum is a global variable shared by the entire program.
- Because each constant exists exactly once, comparing enums with `==` is correct and safe, unlike comparing strings.
- `fromPercent(89.49)` returns `B`, which matches the syllabus scale: an A starts at 89.50.

### Methods every enum gets

**Table 2:** Methods Java provides for every enum, shown with LetterGrade.

| Method | Example result | Notes |
|---|---|---|
| `values()` | `[A, B, C, D, F]` | Static; an array of the constants in declaration order |
| `valueOf("B")` | `B` | Static; case sensitive, so `valueOf("b")` throws an exception |
| `name()` | `"B"` | The constant's name exactly as declared |
| `ordinal()` | `1` for `B` | Position in the declaration, starting at 0 |
| `compareTo(C)` | negative for `A` | Compares by declaration order (see Section 6) |

Avoid storing meaning in `ordinal()`. If someone later reorders the constants, every ordinal changes. Put real data in fields, as `gradePoints` does.

### Enums and switch

A switch expression over an enum can list the constants directly, and if it covers every constant, no `default` is needed. If someone later adds a constant, the compiler will flag every switch that no longer covers all cases.

```
String message = switch (s.letterGrade()) {
    case A, B -> "Strong work";
    case C    -> "Solid";
    case D, F -> "Please see me during office hours";
};
```

## 6. The Comparable Interface

`Collections.sort(list)` and `TreeSet` need to know which of two objects comes first. For `String` and `Integer`, Java already knows. For your own class, you say so by implementing `Comparable`:

```
public interface Comparable<T> {
    int compareTo(T other);
}
```

The rule for `a.compareTo(b)`: return a negative number if `a` comes before `b`, zero if they belong in the same position, and a positive number if `a` comes after `b`. Only the sign matters. The ordering this defines is called the class's *natural ordering*.

Here is the complete `Student` record. Its natural ordering is by name, then by ID, then by percent.

```
public record Student(String id, String name, double percent)
        implements Comparable<Student> {
```

```
    private static int nextId = 1;

    // Compact canonical constructor: validates, then the fields are assigned.
    public Student {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        if (percent < 0 || percent > 100) {
            throw new IllegalArgumentException("percent must be 0 to 100: " + percent);
        }
        name = name.strip();
    }

    // Static factory that hands out the next ID.
    public static Student create(String name, double percent) {
        String id = "S" + nextId;
        nextId++;
        return new Student(id, name, percent);
    }

    public LetterGrade letterGrade() {
        return LetterGrade.fromPercent(percent);
    }

    public Student withPercent(double newPercent) {
        return new Student(id, name, newPercent);
    }

    // Natural order: by name, then by ID, then by percent.
    @Override
    public int compareTo(Student other) {
        int result = name.compareTo(other.name);
        if (result != 0) {
            return result;
        }
        result = id.compareTo(other.id);
        if (result != 0) {
            return result;
        }
        return Double.compare(percent, other.percent);
    }
}
```

## Two habits worth building

**Use the compare helpers, not subtraction.** You will see `return a - b;` in old code and online examples. With `int` values, subtraction can overflow and return the wrong sign for very large or very small numbers. With `double` values, converting the difference to an `int` turns a difference like 0.4 into 0, so two different values look equal. Use `Integer.compare`, `Double.compare`, and `String`'s own `compareTo` instead.

**Keep compareTo consistent with equals.** A `TreeSet` decides whether two objects are duplicates using `compareTo`, not `equals`. If `compareTo` compared names only, two different students who share a name would return 0, and the set would silently keep only one of them. Breaking ties with every component makes `compareTo` return 0 exactly when the record's `equals` returns `true`.

## Sorting students and grades

```
ArrayList<Student> list = new ArrayList<>();
list.add(Student.create("Grace", 84.0));
list.add(Student.create("Alan", 72.5));
list.add(Student.create("Ada", 89.49));
list.add(Student.create("Barbara", 58.0));
Collections.sort(list);
for (Student s : list) {
    System.out.println(s.name() + " " + s.id() + " " + s.letterGrade());
}
```

```
Ada S3 B
Alan S2 C
Barbara S4 F
Grace S1 B
```

Every enum already implements `Comparable`, and its natural ordering is declaration order. Because `LetterGrade` declares `A` first, sorting puts the best grades first:

```
List<LetterGrade> grades = new ArrayList<>(
        List.of(LetterGrade.C, LetterGrade.A, LetterGrade.F, LetterGrade.B));
Collections.sort(grades);
System.out.println(grades);   // [A, B, C, F]
```

That convenience is also a trap: reordering the constants silently changes how every list of grades sorts. Declaration order is part of an enum's behavior, so change it carefully.

## One natural ordering per class

A class gets exactly one `compareTo`. What if you sometimes want students sorted by percent, or by letter grade? That is the job of a separate interface, `Comparator`, which describes an ordering from the outside. Comparators are much easier to write with lambda expressions, so we will pick them up in Week 14.

## 7. Summary

**Table 3:** Summary of the ideas in these notes.

| Idea | Key point |
|---|---|
| **Immutable object** | Its state is fixed once the constructor finishes; changes produce new objects |
| `final` **field** | Assigned exactly once; locks the variable, not the object it refers to |
| **Record** | A compact, immutable data class with constructor, accessors, `equals`, `hashCode`, and `toString` generated |
| **Compact constructor** | Where a record validates and cleans up its data |
| **Enum** | A class with a fixed set of instances; can have fields, a constructor, and methods |
| `Comparable` | Defines one natural ordering; return negative, zero, or positive |

## 8. Try It

1. Write a record `Course(String code, String title, int creditHours)` whose compact constructor rejects a blank code and credit hours outside 0 to 6. Print one, then compare two equal courses with `equals` and with `==`.
2. Add a method `isAtLeast(LetterGrade other)` to `LetterGrade` using `compareTo`. Be careful: `A` comes *before* `B`, so think about which sign means "at least as good."
3. Change `Student.compareTo` so it compares names only. Add two different students with the same name to a `TreeSet`, print the set's size, and explain the result.